



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET



Seminarski rad

**PRIBAVLJANJE OBRISANIH PODATAKA
TEHNIKOM FILE CARVING**

Predmet: Digitalna forenzika

Smer: Računarstvo i informatika

Modul: Softversko inženjerstvo

Student:

Bogdan Mitrović, br. ind. 2028

Mentor:

Prof. dr Bratislav Predić

Niš, oktobar 2026. godine

Sadržaj

1. Uvod.....	3
2. Digitalna forenzika.....	4
2.1. O digitalnoj forenzici	4
2.2. Principi digitalne forenzike.....	4
3. Fajl sistemi.....	6
3.1. Osnovne funkcije fajl sistema.....	6
3.2. Brisanje fajlova	6
3.3. Organizacija podataka na hard diskovima	6
3.4. Sektori i logičko adresiranje	6
3.5. <i>Slack space</i> i fragmentacija.....	7
4. File carving	8
4.1. Definicija.....	8
4.2. Tehnike.....	8
4.3. Izazovi.....	9
4.4. Postojeći alati	10
5. Formati fajlova.....	10
5.1. Opšte o formatima.....	10
5.2. JPEG/JPG.....	12
5.3. PNG	13
6. C alat za <i>file carving</i>	16
6.1. O alatu.....	16
6.2. Korišćenje	16
6.3. Funkcionalnosti.....	18
6.4. Implementaciona rešenja	20
7. Testiranje i evaluacija.....	22
7.1. Generisanje podataka	22
7.2. Poređenje sa postojećim rešenjima	22
8. Zaključak.....	23
9. Reference	24

1. Uvod

U savremenom digitalnom dobu, količina podataka koja se svakodnevno generiše je veća nego ikada ranije tokom istorije. Digitalna forenzika kao naučna disciplina postala je ključna u istragama sajber kriminala, u brojnim poslovnim zadacima, kao i u procesima povraćaja podataka (data recovery). Kada operativni sistem obriše datoteku, on zapravo ne uništava njen sadržaj na hard disku, već isključivo briše pokazivač na nju unutar tabele fajl sistema (npr. MFT kod NTFS-a ili inode kod ext4 sistema), označavajući taj prostor kao slobodan za novi upis. Sve dok taj prostor ne bude fizički popunjen novim podacima ili operativni sistem obriše memoriju TRIM komandom, originalni bajtovi ostaju netaknuti u nedodeljenom prostoru (*unallocated space*).

Tehnika koja omogućava spašavanje ovakvih podataka naziva se *File Carving* (rezbarenje datoteka). Za razliku od standardnih alata za povraćaj podataka koji pokušavaju da rekonstruišu oštećenu tabelu fajl sistema, file carving u potpunosti ignoriše fajl sistem. Ova tehnika se oslanja isključivo na pretragu sirovih binarnih podataka (*raw data*) u potrazi za specifičnim heksadecimalnim potpisima koji označavaju početak (zaglavlje/header) i kraj (podnožje/footer) određenog formata datoteke.

Cilj ovog seminarskog rada je detaljna teorijska i praktična analiza procesa *file carving*-a. Rad opisuje specifičnosti strukture JPEG i PNG formata, identifikuje inženjerske izazove tokom razvoja alata poput ugnježđenih zaglavlja (*thumbnails*), i prikazuje iterativni razvoj softverskog alata napisanog u programskom jeziku C koji implementira netrivialne operacije za ekstrakciju podataka. Rezultati alata su poređeni sa profesionalnim alatima poput Foremost-a i Scalpel-a.

2. Digitalna forenzika

2.1. O digitalnoj forenzici

Digitalna forenzika je grana forenzičkih nauka koja se bavi prikupljanjem, čuvanjem, analizom i prezentovanjem digitalnih dokaza na način koji je prihvatljiv pred sudom ili u internoj istrazi. Koristi se u istragama sajber kriminala i prevara, u kadrovskim i korporativnim sporovima, u odgovoru na bezbednosne incidente, ali i u svakodnevnim situacijama pri oporavku podataka.

Prema medijumu koji se analizira, deli se na više oblasti:

- forenzika diskova i računara se bavi fajl sistemima, obrisanim podacima, artefaktima operativnog sistema, i najbitnija je za ovaj rad;
- forenzika memorije analizira procese, mrežne veze i ključeve u radnoj memoriji;
- mrežna forenzika koristi snimke saobraćaja i evidencije uređaja, takođe ima veze sa *file carving*-om;
- forenzika mobilnih uređaja i cloud forenzika su usko specifične oblasti.

File carving spada u forenziku diskova, ali se primenjuje i na memorijske dumpove i mrežne snimke.

Postupak se prema smernicama NIST-a (SP 800-86) odvija u četiri faze: prikupljanje (kopiranje podataka sa medijuma), ispitivanje (izdvajanje relevantnih podataka), analiza (izvođenje zaključaka) i izveštavanje. Carving pripada fazi ispitivanja.

2.2. Principi digitalne forenzike

Dokaz je vredan samo ako se može pokazati da nije menjan, pa se rad zasniva na nekoliko principa:

1. Originalni medijum se ne menja. Pri pristupu se koristi hardverski blokator upisa (write blocker), jer i običan priključak diska može da izmeni podatke (montiranje, ažuriranje vremena pristupa, ponovno izvođenje žurnala).
2. Rad na forenzičkoj slici podrazumeva pravljenje tačne kopije bit-po-bit (na primer alatom `dd`), a sve analize se rade na kopiji. Ako se kopija ošteti, lako se pravi nova.
3. Provera integriteta heševima je jako bitan princip za ispravn Za original i kopiju računa se kriptografski heš (danas SHA-256, ranije često MD5). Jednake vrednosti dokazuju da su kopija i original identični, a ponovno računanje posle analize dokazuje da analiza nije izmenila sliku.
4. Lanac nadzora (chain of custody) beleži ko je, kada i zašto imao pristup dokazu.

5. Dokumentovanje i ponovljivost nalažu da se svi koraci beleže dovoljno detaljno da drugi ispitivač na istim podacima dobije iste rezultate.
6. Redosled nestajanja (RFC 3227): prvo se prikupljaju podaci koji najbrže nestaju (memorija), a poslednji oni koji najduže ostaju (disk).
7. Zakonitost nalaže da prikupljanje mora biti ovlašćeno, kao i kod dokaza koji nisu digitalne prirode.

Ovi principi utiču i na dizajn alata iz ovog rada. Alat otvara ulaz isključivo za čitanje (*O_RDONLY*), čita sa apsolutnih pomeraja pozivom *pread* i upisuje izdvojene fajlove u poseban direktorijum, pa slika ostaje neizmenjena. Oslanjanje na *O_RDONLY* nije zamena za blokator upisa, zbog čega se alat pokreće nad slikom (poglavlje 6.2).

3. Fajl sistemi

3.1. Osnovne funkcije fajl sistema

Fajl sistem je struktura kojom operativni sistem organizuje podatke na mediju za skladištenje tako da se oni mogu naći, čitati i menjati. Njegove osnovne funkcije su: imenovanje i hijerarhija, odnosno direktorijumi koji povezuju imena fajlova sa njihovim sadržajem; dodela prostora, odnosno odlučivanje koje jedinice medijuma pripadaju kom fajlu; evidencija slobodnog prostora; metapodaci o fajlu (veličina, vremena, vlasnik, prava pristupa); očuvanje konzistentnosti i druge.

Najčešći fajl sistemi su FAT (memorijske kartice i prenosivi mediji), NTFS (Windows), APFS (macOS) i ext4 (Linux). Razlikuju se u detaljima, ali svi razdvajaju metapodatke (gde je fajl) od sadržaja (šta piše u fajlu). Zbog te razdvojenosti carving i postoji kao validna opcija.

3.2. Brisanje fajlova

Kada korisnik obriše fajl, operativni sistem obično ne briše njegov sadržaj. Menja samo metapodatke: fajl se izbacuje iz direktorijuma, a njegov prostor se označava kao slobodan. Sadržaj ostaje na mediju dok ga novi podaci ne prepisu.

Kada se zapis o obrisanom fajlu ponovo iskoristi ili uništi, ostaje samo sadržaj u nedodeljenom prostoru, a to je situacija u kojoj se koristi carving (poglavlje 4). Kod SSD diskova komanda TRIM može fizički obrisati oslobođene blokove, pa nije garantovano da sadržaj ostaje netaknut.

3.3. Organizacija podataka na hard diskovima

Klasični hard disk čuva podatke na magnetnim pločama podeljenim na koncentrične staze, a staze na sektore. Sektor je najmanja jedinica koju disk čita ili upisuje, tradicionalno veličine 512 bajtova, a kod novijih diskova 4096 bajtova. SSD diskovi nemaju ploče, ali računaru se predstavljaju na isti način, kao niz sektora, najviše zbog kompatibilnosti.

3.4. Sektori i logičko adresiranje

Umesto stare adrese cilindar - glava - sektor (CHS), danas se koristi logičko adresiranje blokova (LBA), sektori su numerisani redom od nule, a pomeraj u bajtovima dobija se množenjem broja sektora njegovom veličinom. Isto važi za sliku diska (`dd`): bajt na pomeraju P pripada sektoru $P / 512$. Na primer, pomeraj 10125824 iz test slike jeste sektor 19777, jer je $19777 \cdot 512 = 10125824$.

Fajl sistem grupiše sektore u klaster (blokove), koji su najmanja jedinica dodele prostora. Tipična veličina je 4 KB, odnosno 8 sektora. Fajl sistem počinje fajlove na granicama klastera, mereno od početka particije. Zato, kada se slika ceo disk, poravnanje u odnosu na sliku zavisi od pomeraja particije, a ne mora da bude očuvano ni pri kopiranju.

3.5. *Slack space* i fragmentacija

Veličina fajla retko je umnožak veličine klastera, pa poslednji klaster ostaje delimično popunjen. Neiskorišćeni deo tog klastera je slack space. Na primer, fajl od 10 000 bajtova na klasterima od 4096 bajtova zauzima tri klastera (12 288 bajtova), a slack iznosi 2288 bajtova. U njemu mogu preostati podaci nekog ranijeg fajla, što je forenzički zanimljivo. Takođe je zanimljivo to što se pomoću alata koji rade na niskom nivou u slack space mogu umetnuti manji fajlovi. Slack space treba razlikovati od nedodeljenog prostora, koji čine celi klasteri koji ne pripadaju nijednom fajlu. Ostaci obrisanih fajlova nalaze se u nedodeljenom prostoru, a ne u slack-u.

Fragmentacija nastaje kada slobodan prostor nije neprekidan pa se fajl smešta u više delova (fragmenata). Češća je kod većih fajlova i fajlova koji rastu tokom vremena. Prema Garfinkelovom istraživanju, na tipičnom disku je niska, ali je znatno viša za forenzički važne tipove, poput JPEG slika i e-pošte (poglavlje 4.3). Za carving je to ključno ograničenje, jer alat koji kopira neprekidan opseg bajtova ne može da sastavi fajl čiji su delovi razbacani po disku.

4. File carving

4.1. Definicija

File carving je tehnika oporavka fajlova na osnovu njihovog sadržaja, a ne na osnovu metapodataka fajl sistema koji na taj sadržaj ukazuju (Garfinkel, 2007). Alat prolazi kroz sirove bajtove slike diska, prepoznaje početak fajla po potpisu, određuje gde se fajl završava i izdvaja ga. Fajl sistem u tom procesu ne učestvuje.

Potreba za ovom tehnikom nastaje kada metapodaci nisu dostupni ili se na njih ne može osloniti, među kojima su i sledeći slučajevi:

- fajl je obrisao, a njegov zapis u tabeli fajl sistema je u međuvremenu ponovo iskorišćen za drugi fajl;
- fajl sistem je oštećen ili formatiran ili je nečitljiv MFT;
- ostaci fajlova se nalaze u nedodeljenom prostoru, van bilo kog zapisa;
- analizira se sadržaj koji nema fajl sistem, na primer memorijski dump, snimak mrežnog saobraćaja ili fajl koji u sebi sadrži druge fajlove.

Oporavak zasnovan na fajl sistemu vraća ime, putanju i vremena fajla, ali ne radi kada su metapodaci uništeni. Carving radi i tada, ali vraća samo sadržaj, bez imena i vremena. Zbog toga se ove dve tehnike u praksi koriste komplementarno.

Carving ne uspeva uvek, već pod sledećim uslovima:

1. sadržaj nije prepisan novim podacima (kod SSD diskova komanda TRIM može fizički obrisati oslobođene blokove i popuniti ih nulama);
2. fajl je na disku smešten neprekidno (poglavlje 3.5);
3. format ima prepoznatljiv potpis i strukturu (šifrovani sadržaj ih nema).

Opšti postupak izgleda ovako. Prvo se napravi forenzička slika diska i izračuna njen heš (poglavlje 2.2). Zatim se slika linearno pretražuje u potrazi za potpisima. Za svaki pronađeni potpis (kandidata) određuje se gde se fajl završava, kandidat se proverava, a zatim se opseg bajtova izdvaja u novi fajl čiji se heš beleži.

4.2. Tehnike

Tehnike se razlikuju po tome kako određuju kraj fajla i koliko proveravaju kandidata.

Zaglavlje–podnožje (header/footer) je tehnika u kojoj se kopira sve od potpisa početka do prvog potpisa kraja. Ovo je najjednostavniji i najbrži pristup i još uvek najrasprostranjeniji u alatima kao što su Foremost i Scalpel. Ne radi za formate bez podnožja, a lažna podnožja neretko daju pogrešne fajlove.

Kada se koriste zaglavlje i maksimalna veličina, kopira se unapred zadat broj bajtova, ili do podnožja ako se ono pojavi ranije. Koristi se za formate bez oznake kraja, ali izlaz obično sadrži višak bajtova iza pravog kraja fajla.

Zaglavlje i struktura formata nalažu da se kraj izračunava iz podataka o dužini u samom fajlu, odnosno iz veličine u zaglavlju (BMP, RIFF), iz lanca chunk-ova (PNG) ili iz segmenata (JPEG). Ovaj pristup primenjuje alat iz ovog rada (poglavlje 6).

Validacija objekata, Garfinkel (2007), predlaže višeslojnu proveru kandidata, u kojoj se jeftini testovi izvršavaju prvi, a skupi samo za one koji prežive (dekodiranje sadržaja, provera kontrolnih suma). Cilj je da se lažni kandidati odbace što ranije.

Carving po blokovima je tehnika u kojoj se pretražuju samo granice blokova (klastera), pod pretpostavkom da fajl sistem počinje fajlove na tim granicama. Ubrzava pretragu i smanjuje broj lažnih pogodaka, ali ne radi ako je poravnanje izgubljeno, na primer kod slike diska koja nije poravnata

Carving fragmentisanih fajlova je najrealniji i najzahtevniji primer tehnike koja se koristi u ovoj oblasti. Garfinkel (2007) uvodi bifragment gap carving, za fajl podeljen na dva dela, isprobavaju se sve moguće praznine između dela sa zaglavljem i dela sa podnožjem, a ispravnim se smatra spoj koji prođe validaciju. Pal i Memon razvili su metode za više od dva fragmenta (Parallel Unique Path, SmartCarving), koje blokove sortiraju po tipu, a zatim ih redosledom sastavljaju. Pal, Sencar i Memon (2008) određuju tačku fragmentacije sekvencijalnim testiranjem hipoteza. Mogu se naći i daleko kompleksniji algoritmi za pronalaženje redosleda fragmenata kao što je sciencedirectTODO!

4.3. Izazovi

Fragmentacija je prvi i najočigledniji izazov ovog procesa. Garfinkel je pregledao više od 300 diskova kupljenih na tržištu polovne opreme i ustanovio da je fragmentacija na tipičnom disku niska (ispod 10 %), ali je za forenzički važne tipove podataka, kao što su e-pošta, JPEG i Word dokumenti, znatno viša. Pretpostavka neprekidnosti zato ograničava svaki jednostavni alat.

Lažni pogoci se takođe javljaju kao problem kada god je prepoznavanje obrazaca u pitanju. Kratak potpis se javlja i u podacima koji nisu fajl. Za trobajtni JPEG potpis to je otprilike jednom na 16 MiB (poglavlje 5.1), a kompresovani i šifrovani sadržaj se ponaša skoro kao nasumičan.

Nepouzdana kraj fajla je posledica toga da i podnožje takođe može biti lažno. U testnoj slici u opsegu od oko 1,5 MB između dva JPEG-a pronađeno je 78 pojava para `FF D9`, iako tamo nije bilo nijednog pravog JPEG zaglavlja. Alat koji se oslanja samo na prvo podnožje zato može da vrati potpuno pogrešan opseg.

Ugnježdjeni objekti često imaju potpise više tipova podataka u svom memorijskom opsegu. JPEG fajlovi često sadrže, na primer, JPEG sa umanjenim slikama (poglavlje 5.2). Alat mora da razlikuje novi fajl od dela fajla koji ga obuhvata.

Nedostatak metapodataka usled nekorisćenja operativnog sistema znači da oporavljeni fajlovi nemaju originalna imena ni vremena, pa se ne može utvrditi kada je fajl nastao ili gde se nalazio. Deo tih informacija ponekad se može rekonstruisati iz samog sadržaja (na primer Exif podaci u fotografijama).

4.4. Postojeći alati

Foremost je razvila američka agencija za istrage Air Force Office of Special Investigations, zajedno sa centrom CISSR. Radi po principu zaglavlje–podnožje. Tipovi fajlova, potpisi i maksimalne veličine zadaju se u konfiguracionom fajlu, a rezultati se beleže u izveštaju `audit.txt`. Korišćen je za potrebe izrade ovog rada.

Scalpel je potpuna prerada Foremost-a 0.69, čiji autori (Richard i Roussev, 2005) ciljaju na veću brzinu. Koristi isti princip i konfiguracioni fajl `scalpel.conf`, u kome se za svaki tip navode ekstenzija, osetljivost na velika i mala slova, maksimalna veličina, zaglavlje i, opciono, podnožje. Ima opciju `-q` za izdvajanje samo kada je zaglavlje poravnato na klaster i režim pregleda (`-p`) koji samo ispisuje šta bi bilo izdvojeno. Podrška za fragmentisane fajlove planirana je u verziji koja je u razvoju (scalpel3). Takođe je značajno doprineo razvoju ovog rada zbog dostupnog izvornog koda.

PhotoRec je deo paketa TestDisk (CGSecurity) i predstavlja vrlo intuitivno i komercijalno rešenje za svakodnevne korisnike i njihove potrebe, poput povratka slučajno obrisanih fajlova. Ignoriše fajl sistem, tako da radi i kada je ozbiljno oštećen. Prvo utvrđuje veličinu bloka, čitajući je iz fajl sistema ili tako što u sirovim podacima traži prvih deset fajlova i iz njihovih položaja izračunava veličinu. Zatim čita medijum blok po blok i svaki blok poredi sa ugrađenom bazom potpisa. Ako je moguće, veličinu fajla čita iz zaglavlja i proverava dosledost fajla. Ako je fajl manji od veličine u zaglavlju, odbacuje ga. Nakon uspešnog oporavka vraća se na ranije blokove sa potpisom koji nije uspeo da se oporavi i pokušava ponovo, čime oporavlja neke fragmentisane fajlove. Prema dokumentaciji, podržava više od 480 ekstenzija (oko 300 familija fajlova).

5. Formati fajlova

5.1. Opšte o formatima

Sa stanovišta fajl sistema, fajl je samo imenovani niz bajtova zadate dužine. Značenje tih bajtova određuje format fajla, odnosno dogovor o tome šta se nalazi na kojoj poziciji i kako se to tumači. Formati su opisani javnim specifikacijama (za JPEG to je preporuka ITU-T T.81, odnosno ISO/IEC 10918-1, a za PNG specifikacija konzorcijuma W3C, odnosno ISO/IEC 15948), pa svaki program može da pročita sadržaj bez posredovanja fajl sistema. Upravo ta osobina omogućava file carving, odnosno ako je struktura formata poznata, fajl se može prepoznati i izdvojiti iz sirovih podataka bez ikakvih podataka ili pomoći operativnog sistema.

Većina formata počinje potpisom (*magic number*), kratkim nizom bajtova fiksne vrednosti koji identifikuje format, praćenim zaglavljem sa metapodacima i samim podacima. Potpis je ono što carving alat traži, ali nije dovoljan sam po sebi. Ako se pretpostavi da su bajtovi ravnomerno raspoređeni, potpis dužine X bajtova se javlja u proseku jednom na svakih 256^X bajtova. Za trobajtni JPEG potpis to znači jednom na 16 MB, pa se u slici diska od 1 GB statistički očekuju oko 64 slučajna pogotka. Za osmobajtni PNG potpis to je jednom na 2^{64} bajtova, praktično nikada. Ovo je procena za idealno nasumične podatke, kojima nalikuju kompresovani i šifrovani sadržaji, dok stvarni podaci nisu ravnomerni. Iz toga sledi da kratke potpise treba dopuniti proverom strukture iza njih.

Za carving je najvažnije kako format čuva svoj kraj, jer alat mora da odredi koliko bajtova da kopira. Postoje četiri osnovna načina:

- Marker kraja na fiksnom obliku (JPEG, GIF, PDF). Kraj se traži pretragom, što je jednostavno, ali podložno lažnim pogocima.
- Veličina u zaglavlju (na primer BMP i formati zasnovani na RIFF kontejneru, kao WAV i AVI). Kraj se izračuna direktno, ali vrednost može biti oštećena.
- Lanac blokova sa dužinama (PNG). Kraj se određuje prelaskom sa bloka na blok.
- Bez oznake kraja (običan tekst, sirovi podaci). Preostaje samo heuristika, kao što je maksimalna veličina, što nije dovoljno pouzdano.

Format	Potpis (heksadecimalno)	Kraj fajla
JPEG	FF D8 FF	marker EOI (FF D9)
PNG	89 50 4E 47 0D 0A 1A 0A	chunk IEND, praćenjem lanca dužina
GIF	47 49 46 38 (37 39) 61 ("GIF87a"/"GIF89a")	bajt 3B
PDF	25 50 44 46 2D ("%PDF-")	poslednja oznaka %%EOF
ZIP	50 4B 03 04	zapis End of Central Directory (50 4B 05 06)
BMP	42 4D ("BM")	veličina fajla u zaglavlju (pomerač 2)

Tabela 5.1. Potpisi i načini određivanja kraja nekih formata

Dodatni problem su kontejnerski formati čiji sadržaj može uključivati druge fajlove. Tako JPEG u metapodacima čuva umanjene slike, PDF sadrži ugrađene JPEG slike, a ZIP-based formati (na primer DOCX) sadrže slike u arhivi. Alat koji u sirovim podacima nađe potpis mora da odluči da li on označava novi fajl ili pripada fajlu koji ga obuhvata. Ova dilema je ključna za JPEG i razrađena je u poglavlju 5.2.

Ovaj rad se ograničava na dva najrasprostranjenija formata slika, JPEG i PNG, jer se razlikuju upravo po načinu određivanja kraja. JPEG ima marker kraja koji se traži pretragom, a PNG ima lanac blokova sa dužinama koji se prati. Na taj način su obuhvaćene dve od četiri strategije iz gornje liste.

5.2. JPEG/JPG

JPEG (Joint Photographic Experts Group) je standard za kompresiju digitalnih slika čija je unutrašnja struktura zaglavlja zasnovana na nizu segmenata, označenih specifičnim markerima. Svaka validna JPEG datoteka počinje markerom FF D8 (Start of Image - SOI), a završava se markerom FF D9 (End of Image - EOI). Između ova dva markera nalazi se niz strukturalnih blokova.

Većina segmenata prati strog format koji čine dvobajtni marker (koji uvek počinje bajtom FF) praćen dvobajtnom dužinom segmenta (koja uključuje i sama dva bajta dužine), nakon čega sledi deo koji sadrži potrebne informacije (*payload*). Navedeni su segmenti od visokog značaja za forenzičku analizu JPG fajlova, te i za file carving.

APPn segmenti (*application*) su parovi bajtova od FF E0 do FF EF. APP0 se najčešće koristi za JFIF metapodatke, dok se APP1 masovno koristi za EXIF podatke (informacije o kameri, GPS lokacija) i, što je najvažnije za ovaj rad i forenzičku analizu, ugnježdene umanjene prikaze slike (*thumbnails*).

DQT, SOF i DHT definišu kvantizacione tabele, okvire i Hafmenove (*Huffman*) tabele neophodne za dekodiranje kompresovanih podataka.

SOS (*Start of Scan*) je par bajtova FF DA. Ovaj segment označava početak samih kompresovanih vizuelnih podataka. Segment kompresovanih podataka nema eksplicitnu dužinu, već se proteže sve do EOI markera.

Jedan od većih izazova pri carving-u JPEG formata proističe iz činjenice da APP1 segment često sadrži potpuno validnu, ali minijaturnu JPEG datoteku (Thumbnail) namenjenu za brzi pregled više slika, kao što je to slučaj u *grid view* pregledu foldera koji podržavaju svi

moderni operativni sistemi. Ovo znači da se unutar glavnog fajla nalaze dodatni SOI i EOI markeri.

Na primer, tokom analize testnih podataka dostupnih na repozitorijumu, identifikovano je glavno zaglavlje fotografije na ofsetu 10570240. Međutim, unutar njenih metapodataka, nalazila su se čak dva ugnježdene zaglavlja thumbnail-ova, na ofsetima 10570636 i 10574693. Naivni algoritmi, kao i prve verzije alata opisanog u ovom radu, koji traže prvi FF D9 marker na disku će preseći datoteku na podnožju thumbnail-a. Rezultat je korumpirana slika – u pomenutom primeru, umesto originalne fotografije veličine 444 KB, dobija se potpuno neupotrebljiv fajl veličine svega 4102 bajta čije zaglavlje pripada originalnom fajlu, a podnožje *thumbnail* slici originalnog fajla. Tokom iteracija alata su zabeleženi i slučajevi gde je preskočeno zaglavlje originalnog fajla, te je pronađeno zaglavlje *thumbnail*-a i vraćen je fajl vrlo malih dimenzija koji se regularno prikazuje u svim testiranim aplikacijama.

Važno svojstvo JPEG kompresije je tehnika *byte stuffing*-a. Budući da kompresovani niz podataka (nakon SOS markera) nema jasne celine niti poznate šablone bajtova, postoji teorijska šansa da se u njemu slučajno formira bajt FF. Kako forenzički alat taj bajt ne bi pomešao sa kontrolnim markerom (npr. FF D9), standard nalaže da enkoder nakon svakog kompresovanog FF bajta obavezno umetne 00 (nulu). Zahvaljujući ovom pravilu, postalo je nemoguće da se validan FF D9 marker slučajno pojavi unutar vizuelnih podataka, što FF D9 čini pouzdanim indikatorom kraja datoteke. Sličan princip je u računarstvu široko rasprostranjen, na primer pri korišćenju *escape* karaktera, što 00 na neki način i jeste u ovom slučaju.

5.3. PNG

PNG (*Portable Network Graphics*) je format rasterskih slika sa kompresijom bez gubitaka, nastao sredinom devedesetih godina kao zamena za GIF, a kasnije standardizovan kao ISO/IEC 15948. Podržava providnost (alfa kanal) i različite dubine boja, pa se široko koristi za snimke ekrana, grafiku na veb stranicama i ikone. Za razliku od JPEG-a, čija je struktura okrenuta segmentima sa markerima, PNG je od početka zamišljen kao niz blokova sa eksplicitnim dužinama, što ga čini znatno pogodnijim za carving. U ovom radu će biti primer druge strategije *file carving*-a

Svaki PNG fajl počinje osmobajtnim potpisom 89 50 4E 47 0D 0A 1A 0A. Svaki bajt ima svoju ulogu: 89 ima postavljen najviši bit, pa fajl nije moguće zameniti za tekst i otkriva se prenos koji odbacuje osmi bit; 50 4E 47 su ASCII slova "PNG"; 0D 0A (CR LF) otkrivaju pogrešnu konverziju oznaka novog reda sa DOS na Unix konvenciju; 1A (Ctrl-Z) zaustavlja ispis fajla na DOS sistemima; 0A (LF) otkriva obrnutu konverziju. Potpis je zbog toga neuobičajeno dug i teško se slučajno formira u nasumičnim podacima (poglavlje 5.1).

Iza potpisa sledi niz blokova (*chunk*-ova) sa jedinstvenom strukturom.

Polje	Veličina	Opis
Dužina	4 bajta	broj bajtova u polju podataka (bez samog polja dužine, tipa i CRC-a), big-endian, najviše $2^{31} - 1$
Tip	4 bajta	četiri ASCII slova (A–Z, a–z), na primer IHDR
Podaci	dužina bajtova	sadržaj zavisi od tipa
CRC	4 bajta	CRC-32 izračunat nad poljima tip i podaci

Tabela 5.2. Struktura PNG chunk-a

Najmanji mogući chunk zauzima 12 bajtova (prazno polje podataka). Velika i mala slova u tipu imaju značenje: velika početna slova označavaju kritične chunk-ove koje deko­der mora da razume, dok mala označavaju pomoćne (ancillary) koje može da preskoči. Zahtev da tip čine isključivo slova koristi se u alatu kao jeftina provera ispravnosti.

PNG Standard propisuje četiri obavezna tipa chunk-ova:

- IHDR mora biti prvi chunk i sadrži osnovne parametre slike;
- PLTE je paleta boja (obavezna samo za slike sa indeksiranim bojama);
- IDAT sadrži same piksele, filtrirane i sažete zlib/DEFLATE algoritmom dok podaci mogu biti podeljeni na više uzastopnih IDAT chunk-ova;
- IEND mora biti poslednji chunk i ima prazno polje podataka.

Između njih mogu stajati pomoćni chunk-ovi, na primer tEXt (tekstualni podaci), tIME (vreme izmene) i pHYS (fizičke dimenzije).

Polje	Veličina	Primer (slika 60×60)
Širina	4 bajta	00 00 00 3C (60)
Visina	4 bajta	00 00 00 3C (60)
Dubina boje	1 bajt	08 (8 bita po kanalu)

Tip boje	1 bajt	02 (RGB)
Metod kompresije	1 bajt	00
Metod filtriranja	1 bajt	00
Metod preplitanja	1 bajt	00 (bez preplitanja)

Tabela 5.3. Polja chunk-a IHDR (13 bajtova)

Struktura PNG-a ima nekoliko osobina koje olakšavaju carving:

Kraj se izračunava, a ne traži. Pošto svaki *chunk* deklarira svoju dužinu, početak sledećeg *chunk*-a je poznat, pa se od potpisa do IEND stiže prelaskom sa *chunk*-a na *chunk*. Time se izbegava problem lažnih podnožja koji opterećuje JPEG. Iako je niz od dvanaest bajtova na kraju fiksno, pretraga za njim ne bi proveravala ispravnost strukture, pa se u alatu ne koristi.

Kandidat je ispravan samo ako je prvi chunk IHDR, ako se svaki tip sastoji od četiri slova i ako dužina ne prelazi $2^{31} - 1$. Potpis i tip IHDR zajedno čine dvanaest bajtova fiksne vrednosti, pa se nasumični podaci ovom testu praktično ne prilagođavaju.

Kontrolne sume omogućavaju još strožu proveru, iako se u ovom radu ne koriste. Ispravan CRC-32 u svakom *chunk*-u praktično je nemoguće dobiti u nasumičnim podacima ili na spoju fragmenata. Alat u trenutnoj verziji CRC ne proverava (poglavlje 6.4).

Fragmentisani fajlovi se odbacuju u trenutnoj verziji alata. Ako se na sredini lanca ispostavi da sledeći bajtovi pripadaju drugom fajlu, tip chunk-a će najverovatnije sadržati bajt koji nije slovo, pa se kandidat odbacuje umesto da se izdvoji oštećen fajl.

U poređenju sa JPEG-om, PNG nudi pouzdanije određivanje kraja i jaču proveru ispravnosti. JPEG je, sa druge strane, češći na kamerama i uređajima, pa je u praksi većina forenzički zanimljivih fotografija upravo u tom formatu.

6. C alat za *file carving*

6.1. O alatu

Alat razvijen u okviru ovog rada je konzolni program napisan u programskom jeziku C, koji iz sirove slike diska izdvaja JPEG i PNG fajlove, bez obzira na to da li su obrisani, da li je fajl sistem oštećen ili je disk formatiran. U skladu sa tehnikom opisanom u poglavlju 4, alat ne koristi fajl sistem. Ulazne podatke posmatra kao običan niz bajtova, traži potpise formata, a zatim na osnovu strukture samog formata određuje gde se fajl završava.

Prilikom projektovanja je cilj bio alat koji ispunjava većinu zahteva sličnih postojećih alata, ali jednostavniji. Pored toga što bi alat morao da izdvaja tačne bajtove koji odgovaraju opsegu sadržanom u originalnoj slici, treba da bude i otporan na lažne pogotke, ugnježdene fajlove, da bude čistog i čitkog koda i da fajlovi koje vrati budu validni i sa mogućnošću otvaranja.

Alat obrađuje fajlove koji su na disku smešteni u jednom neprekinutom nizu blokova. Fragmentisani fajlovi nisu podržani, što je detaljnije razmotreno u poglavlju 6.4.

6.2. Korišćenje

Pokretanje alata vrši se iz komandne linije putem jednostavne sintakse:

```
./carver <input_device_path> <output_directory>
```

Tokom izvršavanja, program u terminal ispisuje detaljne logove (dnevnik rada) koji uključuju tačne ofsete na kojima su pronađena zaglavlja, obaveštenja o preskočenim ili korumpiranim strukturama, kao i ukupnu veličinu izvučenih datoteka. Ekstrahovani fajlovi se automatski imenuju inkrementalno, prema šablonu `carved_<index>.<ext>` (npr. `carved_0.jpg`, `carved_1.png`), i smeštaju u definisani izlazni direktorijum.

Alat se prevodi standardnim prevodiocem:

```
gcc -o carver carver.c
```

Kao ulaz se preporučuje slika diska, a ne sam uređaj. Slika se pravi alatom `dd`

```
dd if=/dev/sdX of=input.dd bs=1M
```

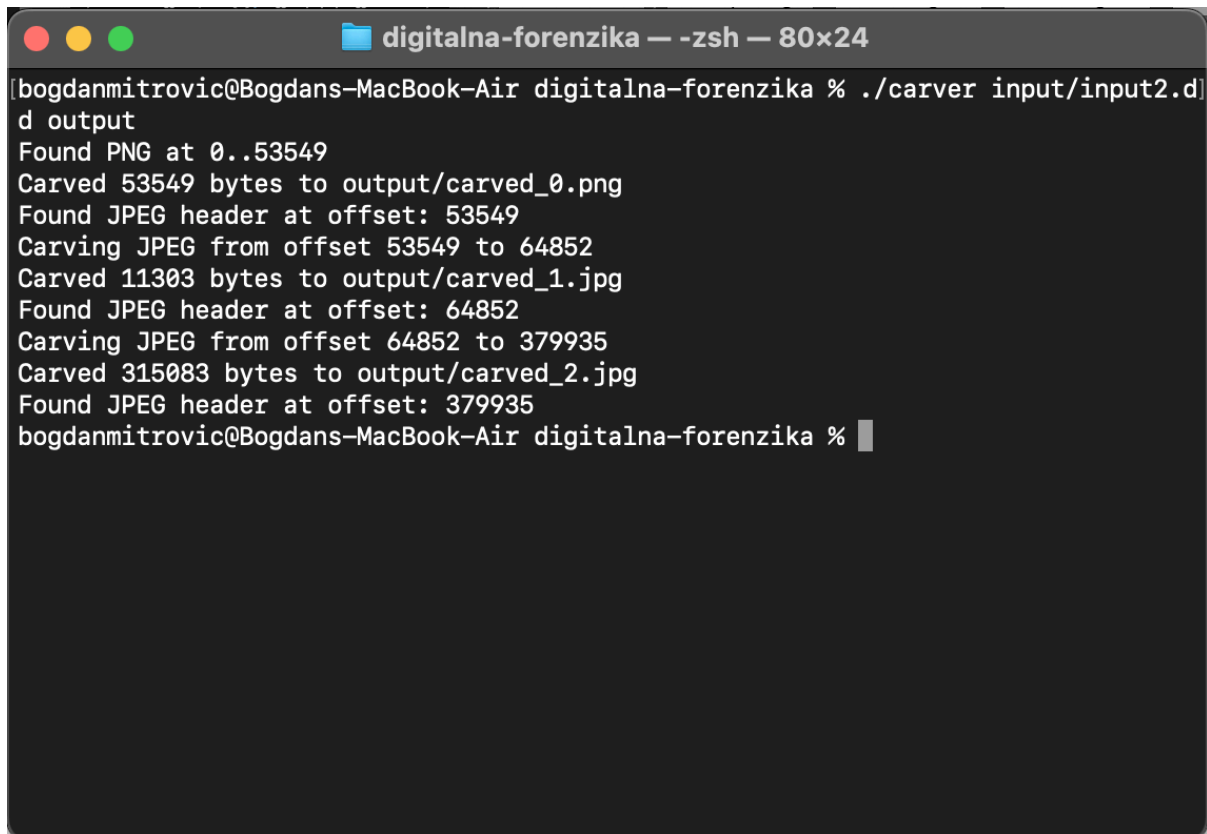
a njen SHA-256 heš može da se izračuna pre i posle obrade da bi se pokazalo da ulaz nije izmenjen. Pokretanje ima sledeći oblik:

mkdir -p output

./carver input/input.dd output

Argument	Značenje
<ulaz>	putanja do slike diska (ili uređaja)
<izlazni_direktorijum>	direktorijum u koji se upisuju izdvojeni fajlovi; mora unapred postojati

Izdvojeni fajlovi dobijaju imena carved_N.jpg odnosno carved_N.png, gde je N redni broj koji počinje od nule i zajednički je za oba formata. Tokom rada alat ispisuje sledeće poruke:



```
[bogdanmitrovic@Bogdans-MacBook-Air digitalna-forenzika % ./carver input/input2.d
d output
Found PNG at 0..53549
Carved 53549 bytes to output/carved_0.png
Found JPEG header at offset: 53549
Carving JPEG from offset 53549 to 64852
Carved 11303 bytes to output/carved_1.jpg
Found JPEG header at offset: 64852
Carving JPEG from offset 64852 to 379935
Carved 315083 bytes to output/carved_2.jpg
Found JPEG header at offset: 379935
bogdanmitrovic@Bogdans-MacBook-Air digitalna-forenzika %
```

Slika x. Izlaz alata u konzoli

Poruka	Značenje
Found JPEG header at offset	pronađeno je zaglavlje koje je prošlo proveru strukture; alat traži podnožje

Carving JPEG from offset A to B	pronađeno je podnožje; opseg [A, B) se kopira u izlazni fajl
Found PNG at A..B	PNG je pronađen i njegov kraj je određen; fajl se odmah izdvaja
Ignoring ... header at N	potpis postoji, ali struktura iza njega nije ispravna, pa se pogodak odbacuje
Carved N bytes to ...	izdvajanje je završeno

Program ima povratnu vrednost 0 ako je obrada završena, a 1 ako se ne može otvoriti ulaz, alocirati memorija, napraviti izlazni fajl ili pročitati ulaz. Rezultati se proveravaju naredbama `shasum -a 256 output/*` (poređenje sa heševima originala, ako postoje) i `file output/*` (da li su izdvojeni fajlovi prepoznati kao ispravni JPEG odnosno PNG). Bitno je napomenuti da na macOS operativnom sistemu neobrađeni uređaji `/dev/rdiskN` zahtevaju čitanje poravnato na veličinu sektora, koje alat ne obezbeđuje, pa se koristi slika diska ili uređaj `/dev/diskN`.

6.3. Funkcionalnosti

Alat pretražuje ulaz u blokovima od 4096 bajtova. Obrada jednog bloka teče na sledeći način: blok se najpre čita pozivom `pread` sa apsolutnog pomeraja `pos`. Ispituje se svaka pozicija na kojoj je dostupno bar 8 bajtova (dužina najdužeg potpisa, PNG-a). Ukoliko se na poziciji nalazi PNG potpis, poziva se `png_walk`. Ako je struktura ispravna, fajl se odmah izdvaja, a pretraga se nastavlja iza njegovog kraja. Ako se nalazi `FF D8 FF`, poziva se `app_segments_end` kako bi se približnije odredilo da li je to legitimno zaglavlje ili lažni pogodak. Ako je struktura ispravna, pamti se početak kandidata, a sve do kraja njegovih metapodataka pozicije se preskaču. Ako se nalazi `FF D9`, a postoji otvoren kandidat za zaglavlje i pozicija je iza njegovih metapodataka, izdvaja se opseg od zaglavlja do kraja podnožja i čuva se kao pronađen fajl. Pomeraj za sledeći blok uvećava se za veličinu bloka umanjenu za 7 bajtova.

Potpis ili podnožje može biti presečeno granicom dva bloka, pa se susedni blokovi preklapaju za 7 bajtova, odnosno za dužinu najdužeg potpisa umanjenu za jedan. Tako se svaka pozicija ispituje tačno jednom i uvek sa svim bajtovima potrebnim za poređenje. Poravnanje fajlova sa granicama sektora ne može se uzeti kao apsolutno tačno, jer podnožja stoje na proizvoljnim pomerajima, a poravnanje se gubi i pri kopiranju slike.

JPEG potpis je FF D8 FF. Kako sam potpis nije dovoljan, jer se neretko slučajno javlja u nasumičnim podacima, pa funkcija `app_segments_end` proverava šta iza njega sledi. JPEG posle SOI markera sadrži niz segmenata oblika marker, dvobajtna dužina, sadržaj. Funkcija prolazi kroz lanac APPn segmenata (FF E0–FF EF), tako što se svaki preskače za deklarisanu dužinu, a lanac mora biti praćen tabelom kvantizacije (DQT), Hafmen (Huffman) tabelom (DHT) ili zaglavljem okvira (SOF). U suprotnom se zaglavljje odbacuje. Podnožje je prvi par FF D9 iza tog lanca. Pošto se u kompresovanim podacima svaki bajt FF prati bajtom 00, FF D9 se tamo ne može slučajno pojaviti. Ako se pre pronalaska podnožja pojavi novo ispravno zaglavljje, ono zamenjuje prethodnog kandidata, čime se napušta fajl čiji kraj nije nađen.

PNG potpis ima 8 bajtova (89 50 4E 47 0D 0A 1A 0A). PNG je niz *chunk*-ova, od kojih svaki ima 4-bajtnu dužinu, 4-bajtni tip, podatke i 4-bajtnu CRC-32 vrednost. Funkcija `png_walk` preskače od *chunk*-a do *chunk*-a po dužini i proverava da je prvi chunk IHDR, da se tip sastoji od četiri slova i da dužina ne prelazi $2^{31} - 1$. Fajl se završava na *chunk*-u IEND. Za razliku od JPEG-a, kraj se ne traži pretragom nego prati lancem dužina, pa nema lažnih podnožja. Kandidati veći od 256 MB odbacuju se.

Sadržaj APPn segmenata može da uključuje kompletne JPEG-ove, najčešće Exif i Photoshop umanjene slike, sa sopstvenim zaglavljima i podnožjima. Promenljiva `skip_until` čuva pomeraj do kog se zaglavljja i podnožja ignorišu, kod JPEG-a to je kraj lanca APPn segmenata, a kod PNG-a kraj celog fajla. Tako se umanjena slika ne izdvaja kao poseban fajl niti prekida izdvajanje fotografije.

Neuspeh otvaranja ulaza, alokacije memorije ili pravljenja izlaznog fajla prekida rad sa porukom i kodom 1. Greška pri čitanju u petlji pretrage takođe prekida rad. Ako čitanje ili pisanje zakaže usred izdvajanja, na standardni tok za greške ispisuje se upozorenje o nepotpunom fajlu, delimični fajl se zadržava (što je u forenzici često korisno), a obrada se nastavlja.

	JPEG	PNG
Potpis	3 bajta (FF D8 FF)	8 bajtova (89 50 4E 47 0D 0A 1A 0A)
Provera strukture	lanac APPn segmenata, zatim DQT/DHT/SOF	IHDR prvi; tipovi od slova; dužina $\leq 2^{31} - 1$
Određivanje kraja	prvi FF D9 iza lanca	praćenje chunk-ova do IEND
Ugnježdjeni objekti	APPn segmenti se preskaču	ceo fajl se preskače
Ograničenje veličine	nema	256 MB
Provera kontrolnih suma	nema (format ih nema)	CRC-32 se ne proverava

Tabela 6.1. Poređenje obrade dva formata

Ograničenja ovakvog pristupa razmatraju se u poglavlju 6.4.

6.4. Implementaciona rešenja

Tokom razvoja alata, prepoznato je i otklonjeno više ograničenja. Za svaki problem primenjena je metodologija identifikacije simptoma, analize uzroka, implementacije softverskog rešenja i empirijske validacije nad identičnim podacima. Pored toga, uklonjene su i greške koje nisu empirijski dobijene, već su pronađene analizom koda i pretpostavkama mogućih nepravilnosti.

Prve pronađene greške su se dešavale nakon prve uspešno sačuvane datoteke. Uzrok je bila greška tokom manipulacije pokazivačem pozicije u fajlu. Radi olakšane manipulacije i sprečavanja daljih grešaka, ali i ubrzanog razvoja u kasnijim fazama alata, kao rešenje su iskorišćeni POSIX sistemski pozivi koji prihvataju apsolutne *offset*-e, čime je omogućeno čuvanje više pokazivača na poziciju u otvorenom fajlu.

Kako je čitanje memorije bajt po bajt neefikasno, novije verzije alata čitaju više bajtova u određeni niz (bafer - *buffer*) koji je prvo povećan na 1KB, a potom i na 4KB kako bi bio usklađen sa veličinom blokova memorije modernih fajl sistema. Iako na prvi pogled trivijalno rešenje, ovaj način učitavanja delova memorije uvodi i novi problem koji se može opisati kao cepanje zaglavlja ili podnožja fajlova. Naime, moguće je teoretski pronaći prvi bajt zaglavlja na kraju jednog 4KB segmenta, a sledeći na početku narednog 4KB segmenta. Kao rešenje se uvodi memorijsko preklapanje u dužini najdužeg poznatog traženog niza bajtova. Iako se zaglavlja javljaju na početku fajla, a fajlovi smeštaju na početak segmenta, postoji šansa da se prilikom kreiranja kopije ili slike diska ili manipulacije diskom desi da poravnanje (*alignment*) bude izgubljeno, stoga je memorijsko preklapanje neophodno.

Alat je kasnije prijavljivao pronalaženje slike, ali rezultujući fajl iznosi 4102 bajta umesto originalnih 444 KB. Uzrok je bilo naivno skeniranje podnožja koje je spajalo početak originalne datoteke (npr. na ofsetu 10570240) sa podnožjem njenog ugnježdenog EXIF *thumbnail*-a (npr. lociranog na 10570636). Primenjeno rešenje je bila implementacija funkcije `app_segments_end` koja čita dvobajtni indikator dužine APPn markera i precizno preskače celokupan blok metapodataka, potpuno ignorišući ugnježdena zaglavlja. Time je postignuto da alat uspešno zaobilazi *thumbnail*-ove i uspešno beleži kompletnu fotografiju od 444 KB do pravog EOI markera.

Jedan od problema je bio i veliki broj lažnih zaglavlja u nasumičnim podacima (*false positives*). Ovakvi podaci ozbiljno prete da ugroze rad alata zbog samog načina rada, naime pri pronalasku validnog zaglavlja, program prelazi u stanje traženja podnožja. Ukoliko zaglavlje nije pravo, može se dogoditi jedan od dva scenarija, fajl sa lažnim zaglavljem i lažnim podnožjem koji je korumpiran i ne može se otvoriti ili fajl sa lažnim zaglavljem i pravim podnožjem, pri čemu je pravo zaglavlje preskočeno tokom traganja za podnožjem.

Rešenje tog problema je rigoroznija provera validnosti zaglavlja. Slučajna pojava sekvence FF D8 FF usled visoke entropije u neraspoređenom prostoru ili unutar drugih kompresovanih fajlova je pokazala da nije dovoljno pročitati samo 3 bajta, već je potrebno proveriti sadržaj nakon njih. Četvrti bajt mora pripadati opsegu od E0 do EF, što znatno smanjuje broj lažnih zaglavlja, u test primeru duplo. Međutim, kasnije verzije imaju još rigoroznije provere uz pomoć *app_segments_end* metode.

Kod PNG fajlova je primenjena sofisticiranija metoda pronalaženja početka i kraja fajla. Nakon pronalaženja početka vrši se čitanje *IHDR chunk*-a i dekodiranje 32-bitne celobrojne dužine koja omogućava precizne, proračunate skokove, potom dekodiranje svakog uzastopnog *chunk*-a i dolazak do IEND podnožja. Matematičkim izračunavanjem podnožja se uklanja mogućnost za pronalaženjem lažnih podnožja i značajno poboljšava i ubrzava ceo proces.

Prilikom sistemskih prepreka, poput nemogućnosti čitanja usled loših sektora (pread vrati negativnu vrednost), algoritam je dizajniran da ispiše grešku na standardni izlaz za greške (stderr) i sačuva parcijalni fajl ukoliko je moguće, čime se ostvaruje kontrolisan tok programa do kraja.

Trenutna arhitektura bazirana je na pretpostavci o kontinuitetu podataka, što znači da fragmentisani fajlovi neće biti uspešno rekonstruisani. Dodatna ograničenja uključuju: nemogućnost parsiranja datoteka čije se podnožje nalazi u poslednjih 7 bajtova ulaznog fajla usled mehanike preklapanja, izostanak provere PNG CRC-32 kontrolnih suma, nepostojanje gornje granice veličine (size cap) za JPEG fajlove, kao i ekstremno retka mogućnost pojave FF D9 sekvence unutar specifičnih nekompresovanih struktura poput kvantizacionih tabela.

7. Testiranje i evaluacija

Cilj testiranja je da se provere sledeće stavke: da li izdvojeni fajlovi tačno odgovaraju bajtovima u slici, da li alat izdvaja lažne ili neispravne fajlove i kako se rezultati porede sa postojećim alatom.

7.1. Generisanje podataka

Slika *imgs.dd* ima [64.979.456] bajtova i SHA-256 vrednost [83585232e908529286f1ff04c43b4d858604875c733183a9e3b44a07ff818d26]. Spisak fajlova u njoj nije poznat, pa su referentne granice fajlova utvrđene ručnom analizom pomoćnim skriptama: *scan.py*, koja ispisuje sve pojave FF D8 FF i FF D9, i *walk.py*, koja prolazi kroz segmente JPEG-a i određuje tačan kraj. U analiziranom opsegu (oko 8,5–10,7 MB) pronađena su četiri JPEG fajla, četiri nasumične pojave FF D8 FF koje ne pripadaju nijednom fajlu (na pomerajima 8700032, 9004904, 9122804 i 9217432), 78 lažnih podnožja FF D9 između prvog i drugog fajla i tri ugnježdene thumbnail-a. Alat je uspešno vratio sve slike pronađene ručno, a SHA suma je ostala nepromenjena. Kontrolna slika je napravljena slika veličine [257.165] sa 3 JPEG i 2 PNG fajla.

7.2. Poređenje sa postojećim rešenjima

Alat je poređen sa Foremost-om na istim slikama, komandama

```
foremost -t jpg,png -i input/input.dd.
```

```
foremost -t jpg,png -i input/imgs.dd.
```

Foremost za JPEG i PNG koristi ugrađene funkcije koje uzimaju u obzir strukturu formata, pa je poređenje izvedeno sa alatom koji je sposoban za istu vrstu obrade. Pri poređenju su uzeti u obzir netaknutost ulaznog fajla koja je proverena SHA sumom, brzina obrade, broj pronađenih fajlova i ispravnost fajlova. Oba alata su dala identične rezultate, pri čemu je bitno napomenuti da nije moguće izmeriti razliku u brzini na setu podataka ove veličine, kao i da su pojedini fajlovi imali *thumbnail* slike u sebi i da su vraćeni uspešno.

8. Zaključak

Oporavak izbrisanih podataka bez oslanjanja na strukturu fajl sistema predstavlja jedan od zahtevnijih procesa u digitalnoj forenzici. Kroz izradu ovog rada, uspešno je demonstrirana teorijska i praktična primena *file carving* tehnika za ekstrakciju JPEG i PNG datoteka iz sirovih memorijskih slika.

Razvoj namenskog softverskog alata u programskom jeziku C jasno je pokazao razliku između naivnog skeniranja i naprednijeg inženjerskog pristupa. Klasična linearna pretraga pokazala se neadekvatnom usled problema sa ivicama bafera, desinhronizacijom I/O operacija i osetljivošću na ugnježdene strukture (*EXIF thumbnails*). Ovi izazovi su uspešno savladani napuštanjem standardne C biblioteke i prelaskom na POSIX systemske pozive (*pread*), korišćenjem preklapajućih memorijskih blokova (*overlapping chunks*) i implementacijom strukturnog parsiranja fajlova. Rešavanje problema ugnjeđenih umanjenih slika preskakanjem APP segmenata kod JPEG-a, kao i matematičko iteriranje kroz IHDR-IEND blokove kod PNG formata, potvrdili su da pouzdanost ekstrakcije direktno zavisi od dubinskog razumevanja anatomije formata.

Iako razvijeni alat pokazuje visoku preciznost i ostvaruje rezultate uporedive sa industrijskim standardima poput alata *Foremost*, on poseduje određena arhitektonska ograničenja. Fundamentalno ograničenje je oslanjanje na pretpostavku o kontiguitetu (sekvencijalnosti) podataka, zbog čega alat ne može uspešno da rekonstruiše fragmentisane datoteke. Takođe, nedostaje implementacija rigorozne matematičke validacije (CRC kontrolnih zbirova kod PNG fajlova) i definisanje maksimalne dozvoljene veličine datoteke za JPEG format radi sprečavanja nekontrolisanog čitanja.

Pravci daljeg razvoja ovog alata i srodnih forenzičkih rešenja moraju biti usmereni ka rešavanju problema fragmentacije kroz heurističke metode i takozvani *Smart Carving* (ili *gap carving*). Pored toga, analiza modernih, terabajtnih i petabajtnih diskova zahteva masovnu paralelizaciju procesa pretrage, što bi se moglo postići prebacivanjem glavnih algoritama za prepoznavanje obrazaca (*pattern matching*) na grafičke procesore (GPU). Konačno, proširivanje podrške na veći broj različitih formata datoteka i omogućavanje direktnog čitanja sa sirovih blok uređaja uz adekvatne bezbednosne zaštite, učinili bi ovakav alat potpuno spremnim za primenu u realnim forenzičkim istragama.

9. Reference

- [1] **IBM Think – Digital Forensics**, <https://www.ibm.com/think/topics/digital-forensics>, poslednja izmena: 18.07.2026.
- [2] **Kingston Technology – Understanding File Systems**, <https://www.kingston.com/en/blog/personal-storage/understanding-file-systems>
Poslednji put posećeno: 18.07.2026.
- [3] **S. Garfinkel**, *Carving contiguous and fragmented files with fast object validation*, Digital Investigation 4S (DFRWS 2007).
- [4] **G. Richard III, V. Roussev**, *Scalpel: a frugal, high performance file carver*, DFRWS 2005.
- [5] **A. Pal, H. Sencar, N. Memon**, *Detecting file fragmentation point using sequential hypothesis testing*, Digital Investigation 5 (2008), S2–S13.
- [6] **A. Pal, N. Memon**, *The evolution of file carving*, IEEE Signal Processing Magazine, 2009.
- [7] **Yanbin Tang, Junbin Fang**, *Recovery of heavily fragmented JPEG*, Science Direct Magazine, 2016. <https://www.sciencedirect.com/science/article/pii/S1742287616300512>